

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



BLM4522 - AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

Vize Teslim Raporu

**Bora KOCABIYIK
21290270**

Öğr. Gör. Enver BAĞCI

21/04/2026

github.com/brckfrc/database-operations-lab

ÖZET

Bu rapor, büyük veri kümeleri üzerinde veritabanı performans optimizasyonu (Proje 1) ile çok kaynaklı kirli verilerin temizlenerek güvenilir bir akış üzerinden hedef sisteme aktarılmasını (Proje 5) kapsayan iki temel laboratuvar çalışmasını içermektedir.

Birinci projede, sentetik olarak devasa boyutlarda üretilmiş bir E-Ticaret veri seti üzerinden, sistemi donanımsal olarak darboğaza sokan ağır veritabanı sorguları analiz edilmiştir. İyileştirme sürecinde; yazılımsal filtre düzeltmeleri ve amaca yönelik gelişmiş indeksleme stratejileri kurularak veritabanının okuma yükü binlerce kat azaltılmış, okuma süreleri milisaniyeler seviyesine düşürülerek dramatik bir hızlanma elde edilmiştir. Ayrıca sistemin önbellek (cache) mimarisinde yaşanan kilitlenmelere çözümler getirilmiş ve veri güvenliğini sağlamak adına farklı departmanlara yönelik yetkilendirme modelleri test edilmiştir.

İkinci projede ise, birbirinden bağımsız ve içeriği deforme edilmiş iki farklı kaynaktan beslenen dağınık bir yapı ele alınarak, kurumsal ölçekte bir Veri Mühendisliği (ETL) akışı tasarlanmıştır. Bu kapsamda ham veriler izole bir hazırlık ortamına çekilerek format hatalarından ve sahte kayıtlardan tamamen arındırılmıştır. Ardından, geçerli ancak kendi içinde mükerrer (çakışan) olan elemanlar, projeye özgü bir "Müşteri Önceliği" iş kuralı gözetilerek matematiksel olarak tekilleştirilmiştir. Temizlenen verilerin asıl hedef tablolara aktarımı sırasında ise işlem güvenliği devreye sokularak; yarıda kesilme hatalarını tolere eden ve istenildiğinde defalarca sorunsuz çalışabilen (idempotent) kontrollü bir yükleme mimarisi kurulmuştur.

Özetle her iki çalışma da, veritabanı yönetiminde sistemleri donanımsal olarak büyütmekten (Scale-Up) ziyade, mimari tasarım desenlerinin ve gelişmiş veritabanı unsurlarının projelere ustalıkla yedirilmesinin başarıyı belirleyen esas faktör olduğunu laboratuvar çıktılarıyla somutlaştırmaktadır.

İÇİNDEKİLER

ÖZET	i
İÇİNDEKİLER	ii
PROJE 1: VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME	1
1. PROJENİN AMACI VE KAPSAMI	1
1.1. Problemin Tanımı	1
1.2. Projenin Hedefi	1
1.3. Performans Problemlerinin Önemi	1
2. KULLANILAN ORTAM VE ARAÇLAR	1
2.1. Donanım ve Geliştirme Ortamı	1
2.2. Veritabanı Yönetim Sistemi	1
2.3. Yönetim ve Görselleştirme Araçları	2
2.4. İzleme ve Ölçüm Araçları	2
3. VERİTABANI TASARIMI	2
3.1. Senaryo Seçimi: E-Ticaret Sipariş Sistemi	2
3.2. Tablolar ve İlişkiler	2
3.3. DDL Özeti	2
3.4. Veri Modelinin Performans Analizi Açısından Uygunluğu	2
4. VERİ SETİ VE SEED SÜRECİ	3
4.1. Sentetik Veri Üretim Yaklaşımı	3
4.2. Tally / Number Table Yaklaşımı	3
4.3. Üretilen Veri Hacmi	3
4.4. Seed Sürecinin Laboratuvar Senaryosundaki Rolü	3
5. BAŞLANGIÇ PERFORMANS ANALİZİ	3
5.1. Baseline Yaklaşımı	3
5.2. Non-Sargable Sorgu Problemi	4
5.3. Gereksiz Kolon Okuma Problemi	4
5.4. Eksik JOIN / Foreign Key İndeksi Problemi	4
5.5. Başlangıç Bulgularının Değerlendirilmesi	4
6. OPTİMİZASYON SÜRECİ	5
6.1. Non-Sargable Sorgunun Düzeltilmesi	5
6.2. İndeksleme Stratejisinin Kurulması	5
6.3. Covering Index Tasarımı	5
6.4. JOIN ve Foreign Key İndekslerinin Eklenmesi	5
6.5. Gereksiz İndeks Değerlendirmesi	5
6.6. Uygulanan Optimizasyonların Teknik Özeti	5
7. İZLEME ARAÇLARI VE ELDE EDİLEN METRİKLER	6
7.1. SET STATISTICS IO, TIME ON ile Sorgu Bazlı Ölçüm	6
7.2. DMV ile Sistem Bazlı İzleme	6
7.3. Execution Plan Analizi	6
7.4. Öncesi / Sonrası Karşılaştırması	7
7.5. Elde Edilen Sonuçların Yorumlanması	7

8. ROL VE YETKİ YÖNETİMİ	8
8.1. Erişim Yönetimi İhtiyacı	8
8.2. Salt-Okunur Rolün Oluşturulması	8
8.3. Test Kullanıcısı ile Yetki Doğrulaması	8
8.4. Veri Bütünlüğü Açısından Değerlendirme	8
9. KARŞILAŞILAN SORUNLAR	8
9.1. Apple Silicon Ortamında MSSQL Çalıştırma Zorluğu	8
9.2. VS Code MSSQL Araç Zinciri Problemleri	8
9.3. Büyük Veri Çıktılarının Terminal Performansına Etkisi	9
9.4. Uygulanan Geçici ve Kalıcı Çözümler	9
10.EK ANALİZ: PARAMETER SNIFFING	9
10.1. Problemin Tanımı	9
10.2. Gözlenen Senaryo	9
10.3. Uygulanan Çözüm	10
10.4. Performans Açısından Değerlendirme	10
11.SONUÇ	10

PROJE 5: VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI	11
1. PROJENİN AMACI VE KAPSAMI	11
1.1. Problemin Tanımı	11
1.2. ETL Sürecinin Hedefi	11
1.3. İş Kuralı: Customer Verisinin Önceliği	11
1.4. Projenin Kapsamı	11
2. KULLANILAN ORTAM VE ARAÇLAR	11
2.1. Veritabanı Yönetim Sistemi	11
2.2. Docker Mimarisi	11
2.3. Geliştirme ve Test Araçları	12
2.4. PostgreSQL Özelliklerinin Tercih Nedenleri	12
3. VERİTABANI VE ŞEMA TASARIMI	12
3.1. Staging Alanı Tasarımı	12
3.2. Hedef Tabloların Tasarımı	12
3.3. Rejected / Quarantine Yapısı	12
3.4. Duplicate / Audit Log Yapısı	12
3.5. ETL Veri Akış Şeması	12
4. ÖZGÜN VERİ SETİ VE SEED SÜRECİ	12
4.1. Kaynak Verinin Seçimi	12
4.2. Dirty Data Üretim Yaklaşımı	13
4.3. Kasıtlı Hata ve Çakışma Enjeksiyonu	13
4.4. Laboratuvar Veri Setinin Oluşturulması	13
5. EXTRACT VE BAŞLANGIÇ DURUMU	13
5.1. CSV Verisinin Staging Alanına Alınması	13
5.2. Başlangıçtaki Veri Kalitesi Problemleri	13
5.3. Ham Verinin Riskleri	13

6. TRANSFORM: VERİ TEMİZLEME	13
6.1. <u>Regex ile Format Doğrulama</u>	13
6.2. <u>Test / Invalid Kayıtların Ayıklanması</u>	14
6.3. <u>Karantina Tablosuna Yönlendirme</u>	14
7. TRANSFORM: VERİ DÖNÜŞTÜRME VE İŞ KURALI UYGULAMASI	14
7.1. <u>UNION ALL ile Geçici Birleştirme</u>	14
7.2. <u>Source Priority Kuralının Tanımlanması</u>	14
7.3. <u>ROW_NUMBER() ile Deduplication</u>	14
7.4. <u>Customer ve Lead Çakışmalarının Yönetimi</u>	14
7.5. <u>Dönüşüm Aşamasının Teknik Özeti</u>	15
8. LOAD: HEDEF TABLOLARA YÜKLEME VE İŞLEM GÜVENLİĞİ	15
8.1. <u>Temiz Verinin crm_contacts_clean Tablosuna Yüklenmesi</u>	15
8.2. <u>Mükerrer Verinin crm_contacts_duplicates Tablosuna Loglanması</u>	15
8.3. <u>Rejected Verinin crm_contacts_rejected Tablosunda Tutulması</u>	15
8.4. <u>Transaction Yapısı</u>	15
8.5. <u>Idempotency ve Yeniden Çalıştırılabilirlik Tasarımı</u>	15
9. ELDE EDİLEN VERİ KALİTESİ RAPORLARI	16
9.1. <u>Quality Report Sorgusunun Amacı</u>	16
9.2. <u>Ölçülen Temel Metrikler</u>	16
9.3. <u>Clean / Rejected / Duplicate Sonuçlarının Analizi</u>	16
9.4. <u>ETL Pipeline Başarısının Değerlendirilmesi</u>	16
10. KARŞILAŞILAN SORUNLAR	17
10.1. <u>PostgreSQL CTE Kısıtlaması</u>	17
10.2. <u>TEMP TABLE Yaklaşımına Geçiş</u>	17
10.3. <u>Çoklu Hedefe Yükleme Probleminin Çözümü</u>	17
10.4. <u>Nihai Mimari Kararın Gerekçesi</u>	17
11. SONUÇ	17
KAYNAKLAR	18

PROJE 1: VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

Bu rapor, "Büyük veritabanlarında performans analizi, sorgu optimizasyonu, indeks yönetimi ve izleme" projesinin araştırma ve geliştirme süreçlerini detaylandırmak üzere hazırlanmıştır. Proje, veri yığınının teorik zorluklarını donanımla değil, doğrudan indeksleme ve SQL optimizasyonu teknikleri ile aşmayı hedeflemektedir.

1. PROJENİN AMACI VE KAPSAMI

1.1. Problemin Tanımı

Büyük hacimli veritabanlarında optimize edilmemiş sorgular ve hatalı mimari kararlar, sistem kaynaklarını (CPU ve I/O) gereksiz yere tüketerek büyük performans darboğazlarına neden olur. Özellikle "Table Scan" operasyonlarına yol açan kural ihlalleri, veritabanı motorunun milyonlarca satırı bellek üzerinden okumasına sebep olmaktadır.

1.2. Projenin Hedefi

Bu laboratuvar senaryosunda hedef; milyonlarca satır barındıran RDBMS şemalarında, kötü yazılmış SQL sorgularının sisteme ölçülebilir şekilde ("Logical Reads" vb. metriklerle) ne kadar yük bindirdiğini kanıtlamak ve ardından performans optimizasyonu teknikleri uygulayarak bu yükü anlamlı ölçüde iyileştirmektir.

1.3. Performans Problemlerinin Önemi

Veritabanı performans problemleri, kurumsal yazılımlarda donanımsal olarak çok masraflıdır ve sunucu kilitlenmelerine (timeout) yol açar. Bu problemlerin yazılım (indexing, sargable syntax) ile önlenmesi, ölçeklenebilir veritabanı yönetiminin temelini oluşturmaktadır.

2. KULLANILAN ORTAM VE ARAÇLAR

2.1. Donanım ve Geliştirme Ortamı

Proje donanım seviyesinde Apple Silicon (M-Serisi) mimarili bir sistemde geliştirilmiştir. Geliştirme standartları ve SQL script düzenlemeleri kaynak kod editörü olarak VS Code üzerinden yürütülmüştür.

2.2. Veritabanı Yönetim Sistemi

Veritabanı motoru olarak Microsoft SQL Server 2022 seçilmiştir. MacOS ARM mimarisinde doğal MSSQL desteği bulunmadığı için süreç Docker container teknolojisiyle yalıtılmış olarak inşa edilmiştir (mcr.microsoft.com/mssql/server:2022-latest x86 emulator tabanında kullanılmıştır).

2.3. Yönetim ve Görselleştirme Araçları

Gerçekçi performans analizleri yapabilmek ve sorguların "Execution Plan" (Yürütme Planı) grafiklerini donanımsal ivmelenme ile hatasız analiz edebilmek için grafiksel istemci olarak DBeaver Community Edition kullanılmıştır.

2.4. İzleme ve Ölçüm Araçları

İzleme ve analizler iki aşamalı yapılmıştır: Sorgu düzeyinde I/O metrikleri ölçmek için `SET STATISTICS IO, TIME ON` kullanılırken, sistem düzeyinde genel bellek ve önbellek analizlerini ölçümlemek için SQL Server yerleşik Dynamic Management Views (DMVs) araçları devreye alınmıştır.

3. VERİTABANI TASARIMI

3.1. Senaryo Seçimi: E-Ticaret Sipariş Sistemi

Performans analizlerinde verilerin çoklu eşleşmelerle çoğaltıldığı "E-Ticaret Sipariş Sistemi" modeli kurgulanmıştır. Bu kurgu, hem veri fazlalığı hem de devasa birleştirme (JOIN) işlemleriyle sistemi yorma kapasitesine sahip klasik bir modeldir.

3.2. Tablolar ve İlişkiler

Sistem 4 temel tablodan oluşturulmuştur:

- customers (Müşteri bilgileri)
- products (Ürün kataloğu)
- orders (Sipariş ana kayıtları)
- order_items (Sipariş içerisindeki ürün kalemleri)

3.3. DDL Özeti

Tablolar arası hiyerarşi ve anahtar eşleşmeleri şu şekildedir:

- customers.customer_id → orders.customer_id (1:N)
- orders.order_id → order_items.order_id (1:N)
- products.product_id → order_items.product_id (1:N)

3.4. Veri Modelinin Performans Analizi Açısından Uygunluğu

Bu yapı, customers > orders > order_items akışında milyonlarca çapraz satır oluşturma kapasitesine sahiptir. "Yabancı Anahtar (Foreign Key)" indekslemelerinin eksikliğini ve yüksek "Logical Read" maliyetlerini simüle etmek için en ideal ve esnek DDL formatıdır.

4. VERİ SETİ VE SEED SÜRECİ

4.1. Sentetik Veri Üretim Yaklaşımı

Sistemi asıl laboratuvar kondisyonuna sokacak dev boyutlarda veri yüklemek adına statik CSV dosyalarından kaçınılmıştır. Veri tamamen SQL motorunun matematiksel gücü ile, sunucu belleği üzerinden dinamik olarak üretilmiştir.

4.2. Tally / Number Table Yaklaşımı

Onyüz binlerce veriyi hızlı türetebilmek için Common Table Expression (CTE) "Tally/Number" matrisi kullanılmıştır.

```
-- Tam script repository'de sql/02_seed_large_data.sql dosyasındadır.  
WITH E1(N) AS (SELECT 1 UNION ALL SELECT 1 UNION ALL SELECT 1),  
E2(N) AS (SELECT 1 FROM E1 a, E1 b),  
E4(N) AS (SELECT 1 FROM E2 a, E2 b)  
INSERT INTO customers (first_name, last_name, email, registration_date)  
SELECT TOP (100000)  
    'FirstName_' + CAST(ROW_NUMBER() OVER(ORDER BY (SELECT NULL)) AS VARCHAR),  
    'LastName_' + CAST(ROW_NUMBER() OVER(ORDER BY (SELECT NULL)) AS VARCHAR)  
-- CROSS JOIN ile katlanarak kayıtlar çoğaltılır.
```

4.3. Üretilen Veri Hacmi

Tally metodu üzerinden saniyeler içinde;

- 100.000 Müşteri kaydı
- 500.000 Sipariş kaydı
- 1.000.000 Sipariş detayı (Items) veritabanına sorunsuz şekilde yüklenmiştir.

4.4. Seed Sürecinin Laboratuvar Senaryosundaki Rolü

Bu 1.6 Milyon satırlık dev yapı; RAM ve CPU bellek limitlerini test edecek kadar ağır, ama aynı zamanda index optimizasyonlarını da saniyeler seviyesinde gösterecek kadar kontrollü dağılmış bir veri piramidi oluşturmuştur.

5. BAŞLANGIÇ PERFORMANS ANALİZİ

5.1. Baseline Yaklaşımı

Ortam sağlandıktan sonra veritabanına, geliştiricilerin sıklıkla düştüğü yazılım hatalarıyla kodlanan ("kötü") sorgular yöneltmiş ve veritabanı motorunun "Baseline" (Başlangıç) tepkisi kaydedilmiştir.

5.2. Non-Sargable Sorgu Problemi

WHERE YEAR(order_date) = 2024 sorgusu sistemi en çok yoran ilk testtir. Filtre kolonunun (order_date) etrafına dahili bir fonksiyon sarmak, bu kolona bağlı mevcut tüm b-tree indeksleri geçersiz (Non-Sargable) kılar.

5.3. Gereksiz Kolon Okuma Problemi

Her sorguda SELECT * kullanım hatası izlenmiştir. Bu durum gereksiz verinin (network paketi ve RAM üzerinden) yersizce taşınmasına yol açarak bir I/O darboğazına sebep olmaktadır.

5.4. Eksik JOIN / Foreign Key İndeksi Problemi

orders tablosunu customers tablosuna bağlayan (customer_id) yabancı anahtar kolonu fiziksel indeks barındırmadığı için, dev boyutlu JOIN operasyonları sırasında tabloların tam okumaya ("Nested Loop Scan") boyun eğmesine neden olmuştur.

5.5. Başlangıç Bulgularının Değerlendirilmesi

Tüm baseline sorguları neticesinde, "Table Scan" kaynaklı fahiş logical reads seviyeleri ölçülmüş ve Execution planlarda devasa ağaç dalları gözlemlenmiştir. Optimizasyon olmadan sistemin donanım yükseltmeleriyle bile kolayca ölçeklenemeyeceği anlaşılmıştır.

order_id	cus	tota	ord
7	24,158	2,789.5	12-19 16:3
9	27,285	2,806.5	08-22 13:1
23	35,199	2,835.5	12-27 15:4
4	61,392	2,834.5	05-20 11:5
47	39,833	1,068.5	12-08 14:4
50	38,067	1,371.5	11-02 05:1
7	34,728	4,841.5	06-29 15:1
64	45,867	2,701.5	16-07 23:0
65	61,571	4,897.5	15-24 00:5
10	92,911	3,615.5	0-20 08:2
71	73,354	4,030.5	11-06 17:1
75	79,641	350.5	10-14 02:1
80	91,685	1,074.5	18-06 08:1
86	92,159	4,678.5	04-12 11:4
90	11,657	4,848.5	11-07 21:4
91	25,483	1,445.5	0-30 23:0
92	21,573	342.5	15-28 15:0
98	89,490	294.5	05-31 19:0
104	1,194	2,772.5	17-05 23:0
110	25,273	3,666.5	14-25 07:5
112	21,372	1,773.5	12-22 08:5
116	61,763	4,030.5	12-09 04:2
122	21,019	2,473.5	14-30 17:2
128	62,626	4,803.5	16-02 17:3
136	70,640	3,088.5	05-16 13:5
142	64,073	3,067.5	18-06 22:0
148	62,834	3,299.5	18-05 01:5
151	4,265	1,626.5	13-07 08:5
159	86,719	2,948.5	17-09 09:3
165	79,739	405.5	01-24 01:2
170	13,802	860.5	12-20 14:5
173	6,149	3,189.5	19-14 03:3
174	23,226	2,180.5	14-20 14:2
182	39,686	113.5	16-15 08:0
183	3,862	3,478.5	15-03 13:0
187	134	3,493.5	07-11 17:2
190	23,567	2,182.5	02-23 11:4

6. OPTİMİZASYON SÜRECİ

6.1. Non-Sargable Sorgunun Düzeltilmesi

Formül kaldırılarak mantıksal argüman aralığına ($\geq$ ve $\leq$) dönülmüştür.

```
SELECT customer_id, total_amount, order_date FROM orders
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01';
```

6.2. İndeksleme Stratejisinin Kurulması

Problemi teşhis ettikten sonra sadece eksik bir indeks değil "doğru" ve "seçiciliği yüksek" indeks mimarisi tasavvur edilerek SQL tablo katmanlarına CREATE NONCLUSTERED INDEX komutları yerleştirildi.

6.3. Covering Index Tasarımı

SQL Server veri okuması sırasında satır tablosuna defalarca gitmesini önlemek (Key Lookup maliyetini silmek) amacıyla istenen ek verileri indekse "INCLUDE" eden mimari uygulanmıştır.

```
CREATE NONCLUSTERED INDEX IX_Orders_Covering
ON orders(order_date) INCLUDE (customer_id, total_amount);
```

6.4. JOIN ve Foreign Key İndekslerinin Eklenmesi

Tablo eşleşmelerini kilitleyen eksiklik giderilerek her iki tablonun bağlanma noktalarına (örn. customer_id sütunlarına) açık indeks yaratılmış ve JOIN operasyonu hızlandırılmıştır.

6.5. Gereksiz İndeks Değerlendirmesi

İndeks eklemek okuma (SELECT) performansını hızlandırırken, yazma (INSERT/UPDATE/DELETE) işlemlerine ekstra yük oluşturur. Optimizasyon yapılırken "Ezbere her yere indeks atmama" kuralına sadık kalınmış, sadece seçiciliği (selectivity) yüksek kritik kolonlara indeks uygulanmış, sisteme değer katmayacağı görülen gereksiz indeksler DROP atılarak yazma hızı ferahlatılmıştır.

6.6. Uygulanan Optimizasyonların Teknik Özeti

Planlanıp uygulanan tüm indeksler veritabanını taramaktan ("Scan") arama yapmaya ("Seek") başarılı bir şekilde dönüştürmüş, "Nested Loop" engelleri "Hash Match" aşamasına aktararak CPU yükünden arındırılmıştır.

7. İZLEME ARAÇLARI VE ELDE EDİLEN METRİKLER

7.1. SET STATISTICS IO, TIME ON ile Sorgu Bazlı Ölçüm

Optimizasyon işlemleri ezbere yapılmamış, SET STATISTICS IO, TIME ON çıktısıyla desteklenmiştir.

Sorgu 1 (Yıl Filtresi) İçin STATISTICS IO Konsol Çıktısı Karşılaştırması:

- Optimizasyon Öncesi (Table Scan Tıkanıklığı)
Table 'orders'. Scan count 1, logical reads 3039, physical reads 0... SQL Server Execution Times: CPU time = 46 ms, elapsed time = 48 ms.
- Optimizasyon Sonrası (Index Seek)
Table 'orders'. Scan count 1, logical reads 3, physical reads 0... SQL Server Execution Times: CPU time = 0 ms, elapsed time = 1 ms.

7.2. DMV ile Sistem Bazlı İzleme

SQL Server yerleşik Yönetim Görünümleri (Dynamic Management Views) ile sistem çapında izleme sağlanarak sorguların toplam CPU maliyeti raporlanmıştır.

```
-- Örnek DMV Sorgusu: En maliyetli 10 sorguyu tespit etme
SELECT TOP 10
    qs.total_elapsed_time,
    qs.total_logical_reads,
    qs.execution_count
FROM sys.dm_exec_query_stats qs
ORDER BY qs.total_logical_reads DESC;
```

7.3. Execution Plan Analizi

"Execution Plan" grafikleri DBeaver'dan görsel olarak incelenmiş, kırmızı uyarılı kalın oklardan oluşan darboğaz kanıtları yeşil "Seek" oklarına dönüştüğü anlaşılabilir screenshots/klasöründe referanslanmıştır.

7.4. Öncesi / Sonrası Karşılaştırması

Senaryo / Problem	Optimizasyon Öncesi Maliyet	Optimizasyon Sonrası Maliyet	Çözüm
Yıl Filtresi	3039 Logical Read, 48 ms yürütme süresi.	3 Logical Read, 1 ms süresi, Index Seek.	Non-Sargable filtre düzeltildi.
Gereksiz SELECT	Bellek tıkanması, ana tabloya sürekli Key Lookup.	Tablo okunmadan sonuç doğrudan fihrist (Index) içinden döndü.	Covering Index kullanıldı.
Eksik JOIN İndeksi	Nested Loop tarama, yüksek CPU tüketimi.	Optimal Hash Match birleştirme.	Yabancı Anahtar (FK) İndeksi Eklendi.

7.5. Elde Edilen Sonuçların Yorumlanması

Bu sonuçlar, veri mühendisliğinde donanımın her şey olmadığını; ufak bir syntax düzeltmesinin ve doğru indekslemenin, sorgu maliyetini dramatik biçimde (3039 kat daha az okuma yaparak) düşürebileceğini somut ifadelerle kanıtlamıştır.

8. ROL VE YETKİ YÖNETİMİ

8.1. Erişim Yönetimi İhtiyacı

Veritabanı optimizasyonu yalnızca kod yazmak değil; analiz amaçlı ağır raporlama çeken Pazarlama veya Raporlama ekiplerinin yanlışlıkla üretim veritabanındaki ana verileri manipüle etmesini (DELETE/UPDATE) de engellemeyi gerektirir.

8.2. Salt-Okunur Rolün Oluşturulması

Aşağıdaki komutlarla kısıtlı ve güvenilir bir rol atanmıştır:

```
-- report_reader adında özel bir rol oluşturuldu
CREATE ROLE report_reader;
-- Sadece okunmasına izin verildi
GRANT SELECT ON customers TO report_reader;
GRANT SELECT ON orders TO report_reader;
-- Veri ekleme ve silme hakları özel olarak (DENY) kilitlendi
DENY INSERT, UPDATE, DELETE ON orders TO report_reader;
```

8.3. Test Kullanıcısı ile Yetki Doğrulaması

Oluşturulan rolde bir sanal oturum (EXECUTE AS USER = 'test_user') açılarak yeni bir sipariş kaydı atılmaya çalışılmış ve rol güvenliğinin aktif olup olmadığı sorgulandığı vakit sistem hedeflendiği gibi hata (The INSERT permission was denied on the object) vermiştir.

8.4. Veri Bütünlüğü Açısından Değerlendirme

Veri tabanının güvenlik kalkanında (Access Control), departmanlara limitli yetkilendirmeler yapılarak verilerin güvenli bir ortamda optimizasyon raporlarına girebileceği kanıtlanmıştır.

9. KARŞILAŞILAN SORUNLAR

9.1. Apple Silicon Ortamında MSSQL Çalıştırma Zorluğu

Bütün bu optimizasyonların donanım uyumsuzluğu yüzünden MacOS M serisi işlemcilerde engellenme riski ortaya çıktı.

9.2. VS Code MSSQL Araç Zinciri Problemleri

MacOS'da çalışan "SQL Tools Service", x86 emülasyonu sağlanan SQL Engine üzerinde donarak "Execution Plan" araçlarını bloke etmiştir.

9.3. Büyük Veri Çıktılarının Terminal Performansına Etkisi

1.6 milyon satırdan dönen I/O testi çıktılarının, veriyi terminale satır satır basması (STDOUT yükü) nedeniyle gerçek performans analizinin test aracının (IDE) kasması sonucu yanlış ölçülebildiği saptandı.

9.4. Uygulanan Geçici ve Kalıcı Çözümler

Uzantı kaynaklı görselleştirme sorunu direkt olarak DBeaver istemcisine geçilerek kökünden çözüldü. Terminal yükü kaynaklı sahte tıkanıklıklar ise SELECT ... INTO #temp taktikleri uygulanıp çıktılar ekrandan gizlenerek sadece salt veritabanı okuma eforu ölçümlendi.

10. EK ANALİZ: PARAMETER SNIFFING

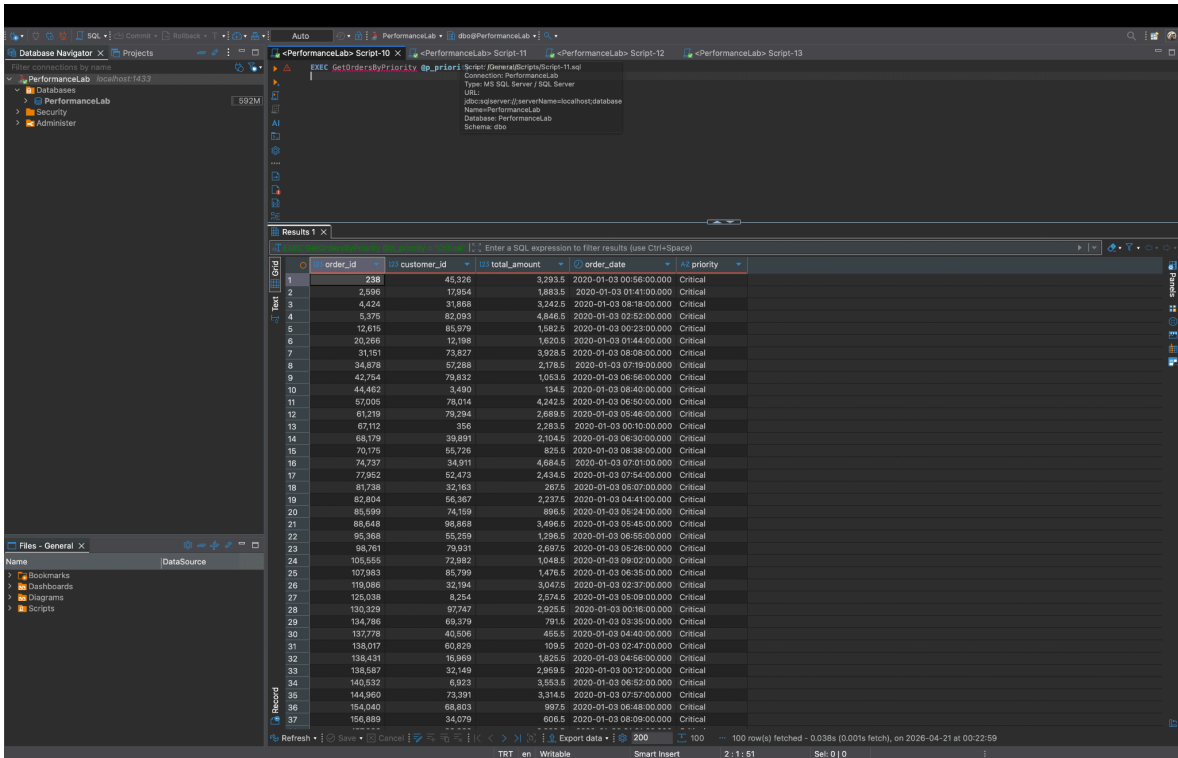
10.1. Problemin Tanımı

Bir diğer inceleme alanı (Ek Çalışma) saklı yordamların (Stored Procedure) plan önbellege (Cache'e) alma problemi olan Parameter Sniffing konusudur.

10.2. Gözlenen Senaryo

Aynı yordam az veri gerektiren 'Critical' parametresi için indeks planlamış; ardından yüzbinlerce veri barındıran 'Normal' parametresi için çağrıldığında o eski önbelleg planını kullanarak hatalı arama tepkisi verip performans sorunlarına yol açmıştır.

- EXEC GetOrdersByPriority @p_priority = 'Critical'; -- 100 Kayıt (Dar plan yapıldı)
- EXEC GetOrdersByPriority @p_priority = 'Normal'; -- Eski dar plan tekrarlandı, performans çok ciddi düştü.



The screenshot shows the DBeaver SQL editor with a query window titled 'PerformanceLab Script-10'. The query is 'EXEC GetOrdersByPriority @p_priority = 'Critical''. The results are displayed in a grid view with columns: order_id, customer_id, total_amount, order_date, and priority. The results show 100 rows of data, all with a priority of 'Critical'. The status bar at the bottom indicates '100 row(s) fetched - 0.038s (0.001s fetch), on 2020-04-21 at 00:22:59'.

order_id	customer_id	total_amount	order_date	priority
238	45,326	3,293.5	2020-01-03 00:56:00.000	Critical
2,696	17,954	1,883.5	2020-01-03 01:41:00.000	Critical
4,424	31,868	3,242.5	2020-01-03 08:18:00.000	Critical
5,375	82,093	4,846.5	2020-01-03 02:52:00.000	Critical
12,615	85,979	1,582.5	2020-01-03 00:23:00.000	Critical
20,266	12,198	1,620.5	2020-01-03 01:44:00.000	Critical
31,151	73,827	3,928.5	2020-01-03 08:08:00.000	Critical
34,878	57,288	2,178.5	2020-01-03 07:19:00.000	Critical
42,754	78,822	1,038.5	2020-01-03 06:46:00.000	Critical
44,462	3,490	134.5	2020-01-03 08:40:00.000	Critical
57,005	78,014	4,242.5	2020-01-03 06:50:00.000	Critical
61,219	79,294	2,688.5	2020-01-03 05:46:00.000	Critical
72,952	356	2,233.5	2020-01-03 00:10:00.000	Critical
68,179	39,891	2,104.5	2020-01-03 06:30:00.000	Critical
70,175	65,726	825.5	2020-01-03 08:38:00.000	Critical
74,737	34,911	4,684.5	2020-01-03 07:01:00.000	Critical
77,952	52,473	2,434.5	2020-01-03 07:54:00.000	Critical
81,738	32,163	267.5	2020-01-03 05:37:00.000	Critical
82,804	66,367	2,237.5	2020-01-03 04:41:00.000	Critical
85,599	74,159	896.5	2020-01-03 05:24:00.000	Critical
88,648	98,898	3,486.5	2020-01-03 05:45:00.000	Critical
95,368	55,259	1,295.5	2020-01-03 05:55:00.000	Critical
98,761	79,931	2,697.5	2020-01-03 05:26:00.000	Critical
105,555	72,982	1,048.5	2020-01-03 09:02:00.000	Critical
107,983	85,799	1,476.5	2020-01-03 06:35:00.000	Critical
119,086	32,184	3,047.5	2020-01-03 02:37:00.000	Critical
125,038	8,254	2,574.5	2020-01-03 05:59:00.000	Critical
130,329	57,747	2,925.5	2020-01-03 00:16:00.000	Critical
134,796	69,279	791.5	2020-01-03 03:35:00.000	Critical
137,772	43,896	455.5	2020-01-03 04:40:00.000	Critical
138,017	60,829	109.5	2020-01-03 02:47:00.000	Critical
138,431	16,969	1,825.5	2020-01-03 04:56:00.000	Critical
138,587	32,149	2,968.5	2020-01-03 00:12:00.000	Critical
140,532	8,823	3,553.5	2020-01-03 06:30:00.000	Critical
144,960	73,391	3,314.5	2020-01-03 07:57:00.000	Critical
154,040	68,803	997.5	2020-01-03 06:48:00.000	Critical
156,889	34,079	606.5	2020-01-03 08:09:00.000	Critical

10.3. Uygulanan Çözüm

Kök sorun derleme koduna OPTION (RECOMPILE) komutu zerk edilip, prosedürün "Eski planı unut ve duruma göre yeni plan çiz" olarak kodlanmasıyla çözüldü.

10.4. Performans Açısından Değerlendirme

Caching mantığının aslında optimizasyon olması beklenirken veri dağılımdaki bir eşitsizlik (skewness) sonucu dev bir handikapa dönüşebildiği açıkça kanıtlanmıştır.

11. SONUÇ

Laboratuvar boyunca doğru tablo mimarisine bağlı Yabancı Anahtar indekslemelerinin hayat kurtarıcı olduğu; veriyi işe yarar kılan en büyük unsurun Sargable Filtre yetkinliği olduğu öğrenilmiştir.

Yapılan B-Tree planlaması, Covering indeksleri ve yazma yükünü dikkate alan bilinçli indeks atılımlarıyla 50 milisaniye aşan yüksek ve tehlikeli logical reads (Table Scan) yükleri; sistem yorulmadan sadece Index Seek kullanan 1-2 ms'lik maliyetlere dönüştürülmüştür.

Proje tamamen veritabanı performans yönetimindeki yazılımsal ve stratejik kararların, SQL donanımlarının önüne geçtiğini açıkça vurgulayan ve belgelenen kanıtlarla sonuca ulaştıran kapsamlı bir çalışma olmuştur.

PROJE 5: VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIMI

Bu rapor, çok kaynaklı kirli veri entegrasyonunun laboratuvar ortamında kontrollü bir senaryo ile temizlenmesi, işlenmesi ve standartlaştırılarak hedef veritabanına aktarılması (ETL) işlemlerini adım adım detaylandırmaktadır.

1. PROJENİN AMACI VE KAPSAMI

1.1. Problemin Tanımı

Modern veritabanı süreçlerindeki en büyük organizasyonel boşluk, farklı departmanların ürettiği çelişkili, düzensiz formatlara sahip ve mükerrer kayıtların aynı havuza girmesidir. Standartlıktan uzak veriler analiz, CRM yönetimi ve performans kalitesini bozan en büyük unsurdur.

1.2. ETL Sürecinin Hedefi

Bu projenin misyonu, birbirine zıt yapıdaki CSV kaynak dosyalarını güvenli bir "Staging" (Hazırlık) alanında harmanladıktan sonra filtrelemek, iş kurallarından geçirerek dönüştürmek ve nihayetinde temiz bir veritabanı nesnesine taşımaktır.

1.3. İş Kuralı: Customer Verisinin Önceliği

Veriler aynı çatı altında birleşirken mükerrer olma (Aynı e-posta adresine sahip birden fazla kayıt) durumu oluşmaktadır. Burada sistem mantığı gereğince "Customer (Satış/Müşteri) verisi, daima Lead (Pazarlama/Liste) verisinden daha büyük önceliğe sahiptir" iş kuralı (Business Rule) benimsenmiştir.

1.4. Projenin Kapsamı

Proje yalnızca hatalı kayıt silimini değil; iş kurallarına göre tekilleştirme (Deduplication) ve elenen mükerrer verilerin Denetim (Audit) amacıyla loglanması gibi profesyonel ETL aşamalarını kapsamaktadır.

2. KULLANILAN ORTAM VE ARAÇLAR

2.1. Veritabanı Yönetim Sistemi

Veritabanı motoru olarak PostgreSQL 16 sistemi (açık kaynaktaki gücü nedeniyle) tercih edilmiştir.

2.2. Docker Mimarisi

Projenin bağımsızlığını korumak adına kurulumlar bir docker-compose alt yapısında izole konteynerlar marifetiyle ayağa kaldırılmış ve proje dizini (etl_db hacmi) sisteme monte edilmiştir.

2.3. Geliştirme ve Test Araçları

İlgili tüm veri yüklemeleri sistem IDE'si (VS Code), veri validasyonu ve tablolar arası analizler PostgreSQL psql (CLI) ile DBeaver Community araçları vasıtasıyla kodlanmıştır.

2.4. PostgreSQL Özelliklerinin Tercih Nedenleri

Veri temizliğinde temel rol oynayan Regex (~) fonksiyonları ve tekilleştirmede (Dedup) kritik rol üstlenen Window Function'lara (ROW_NUMBER() OVER) sahip güçlü ve çevik sentaksı, PostgreSQL'in tercihindeki başat nedendir.

3. VERİTABANI VE ŞEMA TASARIMI

3.1. Staging Alanı Tasarımı

Dışarıdan gelen verinin orijinal yapısı bozulmadan sadece veritabanına girebilmesi adına en ilkel hali olan Staging (Hazırlık) tabloları (stg_customers ve stg_leads) oluşturulmuştur.

3.2. Hedef Tabloların Tasarımı

Her veri kendi kalitesine göre farklı kanallara ayrılmıştır. Süzgeçten başarıyla geçen ilk parti veriler, sistemin asıl temiz tablosuna (crm_contacts_clean) atılmıştır.

3.3. Rejected / Quarantine Yapısı

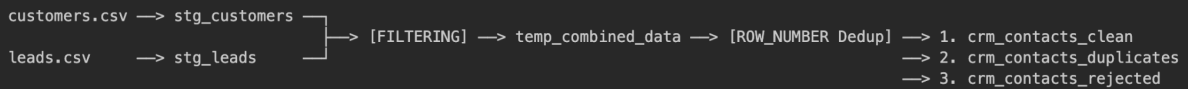
Geçersiz e-posta formatı ve sahte (invalid) içerik saptanan veriler, temiz tabloyu kirletmemeleri için Karantina statüsündeki (crm_contacts_rejected) tabloya sürülerek izole edilmiştir.

3.4. Duplicate / Audit Log Yapısı

Tekilleştirme yarışını öncelik (Priority) kuralını takiben kaybeden temiz yedek veriler sistemden silinmeyip crm_contacts_duplicates log tablosuna atılmış, böylece kurumlar arası Denetim/Audit kuralları korunmuştur.

3.5. ETL Veri Akış Şeması

Projeyi temsil eden akış aşağıdaki şemada betimlenmiştir:



4. ÖZGÜN VERİ SETİ VE SEED SÜRECİ

4.1. Kaynak Verinin Seçimi

Gerçek dışı hazır "Temiz" bir set yerine, rastgele üretilmiş ve iki farklı CSV'ye ayrılmış (Datablist kaynaklı) 100'er satırlık ham veriler projenin temeli seçilmiştir.

4.2. Dirty Data Üretim Yaklaşımı

Projenin "Temizleme" ayağını laboratuvar ortamında somutlaştırmak için ham veriye kasti (make_dirty.sql) operasyonlar enjekte edilerek bozulması sağlanmıştır.

4.3. Kasıtlı Hata ve Çakışma Enjeksiyonu

- UPDATE stg_customers SET email = REPLACE(email, '@', '') WHERE id % 10 = 0;
- UPDATE stg_leads SET first_name = 'Test User', email = 'invalid_email' WHERE id IN (5, 12, 18);

Yukarıdaki algoritmayla formata uymayan '@' eksik e-postalar ve test takılı manipülatif girdiler laboratuvara kasten bulaştırılmıştır.

4.4. Laboratuvar Veri Setinin Oluşturulması

Yapılan de-formasyon işlemi sonucu, "çok kaynaklı kirli veri entegrasyonu"nun kontrollü bir örneği kurularak gerçek hayattaki kirli CRM problemleri kurgulanmıştır.

5. EXTRACT VE BAŞLANGIÇ DURUMU

5.1. CSV Verisinin Staging Alanına Alınması

Dış veri, PostgreSQL komutlarıyla ham hazırlık alanına Çekilmiştir (Extract):

```
\copy stg_customers FROM 'data/source/customers_seed.csv' WITH  
(FORMAT csv, HEADER true)  
\copy stg_leads FROM 'data/source/leads_seed.csv' WITH (FORMAT  
csv, HEADER true)
```

5.2. Başlangıçtaki Veri Kalitesi Problemleri

Staging alanlarındaki 205 adet veriye bakıldığında, bazı e-postaların düz metin ('invalid_email') şeklinde, bazılarının çift kayıt yapısında olduğu saptanmıştır.

5.3. Ham Verinin Riskleri

Standart bir e-posta desenine sahip olmayan yahut kasten 'test' girdisi atanmış bu formların mevcut yapılarla aynı hedef tabloya Merge (Böl/Birleştir) edilmesi büyük veri kalitesi hatalarına ve çifte e-posta yollama skandallarına sebep olacaktır.

6. TRANSFORM: VERİ TEMİZLEME

6.1. Regex ile Format Doğrulama

İlgili tablolardaki ilk eleme, PostgreSQL Regular Expression (Düzenli İfade) süzgeçleriyle yürütülmüştür. Bu desen [A-Za-z0-9._%~!&*@-]+@[A-Za-z0-9._%~!&*@-]+.[A-Za-z]+ şeklinde kullanılmıştır.

6.2. Test / Invalid Kayıtların Ayıklanması

Regex desenini geçemeyen (!~) format bozucu kayıtlarla birlikte sistemi zehirleyen ILIKE '%test%' türünden kasten atılmış bot kaydı benzeri "Spam" veriler yakalanarak ayıklanmıştır.

6.3. Karantina Tablosuna Yönlendirme

Ayıklanan veriler silinmeyip incelenmek ve denetlenmek amacıyla crm_contacts_rejected alanına yönlendirilmiştir:

```
INSERT INTO crm_contacts_rejected (source_record_id,
source_type, ...)
SELECT id, 'Customer', ... FROM stg_customers
WHERE email !~ '^[A-Za-z0-9._%~]+@[A-Za-z0-9.-]+[.][A-Za-z]+$'
OR email ILIKE '%test%' OR email ILIKE '%invalid%';
```

7. TRANSFORM: VERİ DÖNÜŞTÜRME VE İŞ KURALI UYGULAMASI

7.1. UNION ALL ile Geçici Birleştirme

Elenenlerin ardından kalan veriler (stg_customers ve stg_leads içindeki temiz kaynaklar), aynı e-posta üzerinden eşleşme tehlikesini çözebilmek adına PostgreSQL RAM hafızasında yaşatılan geçici bir tabloda (Temp Table) UNION ALL ile dikey olarak sırlanmıştır.

7.2. Source Priority Kuralının Tanımlanması

Birlikte yığılmış sistemde satır statülerini temsil eden source_priority kuralı yazıldı: Müşteriler (1), Satışlar (2) potansiyel derecesi puanını aldı.

7.3. ROW_NUMBER() ile Deduplication

Window functions sayesinde, birebir aynı e-posta grubuna düşen (PARTITION BY email) satırlar içerisinde puanlara göre bir dedupasyon zıplaması dizayn edildi.

7.4. Customer ve Lead Çakışmalarının Yönetimi

```
SELECT source_record_id, source_type, first_name, email,
ROW_NUMBER() OVER (PARTITION BY email ORDER BY
source_priority ASC) as row_num
FROM temp_combined_data;
```

7.5. Dönüşüm Aşamasının Teknik Özeti

Oluşturulan row_num = 1 değerine sahip kayıtlar, yarıştaki asıl kayıtlardır (Clean). Satır değeri 1'den büyük kalanlarsa Customer önceliğine elenmiş olan aynı kişi adresi verileridir (Duplicates).

8. LOAD: HEDEF TABLOLARA YÜKLEME VE İŞLEM GÜVENLİĞİ

8.1. Temiz Verinin crm_contacts_clean Tablosuna Yüklenmesi

Hataları silinmiş ve öncelik sırasında önde kalmayı başarıp row_num = 1 seçilmiş birincil kıymetli veriler:

```
INSERT INTO crm_contacts_clean (first_name, last_name, email,
data_source)
SELECT first_name, last_name, email, source_type
FROM temp_combined_data WHERE row_num = 1;
```

8.2. Mükerrer Verinin crm_contacts_duplicates Tablosuna Loglanması

Aynı isimde olup öncelik sıralaması (row_num > 1) sebebiyle elenen audit verileri silinmek yerine:

```
INSERT INTO crm_contacts_duplicates (original_record_id, email,
issue)
SELECT source_record_id, email, 'Yüksek öncelikli Customer kaydı
mevcut'
FROM temp_combined_data WHERE row_num > 1;
```

8.3. Rejected Verinin crm_contacts_rejected Tablosunda Tutulması

Bozuk yapıya hitap eden Reddedilmiş verilerin tutulduğu tablo yapısı Load aşamasıyla beraber tam olarak test edilmiş ve doğrulanmıştır.

8.4. Transaction Yapısı

Etkileyici olan tüm süreci tek blokta buluşturan mimaridir. Yukarıdaki Extract, Filter, Dedup ve Load hamleleri tabanı kirletmemesi için bir blok transaction olarak BEGIN; ... COMMIT; sarmalı içerisinde yer almıştır.

8.5. Idempotency ve Yeniden Çalıştırılabilirlik Tasarımı

Sürecin herhangi bir noktasında meydana gelebilecek bir güç hatası sistemde yarım kayıtlar bırakmasın diye ROLLBACK ihtimalleri gözetilmiştir. Her ETL işlem operasyonu öncesi hedeflerdeki log tabloları silinir (TRUNCATE), bu durum scripti defalarca aynı sonuçla çalıştırabilme standardıdır.

9. ELDE EDİLEN VERİ KALİTESİ RAPORLARI

9.1. Quality Report Sorgusunun Amacı

Çalışan senaryonun matematiksel olarak "kayıp veri" ile sonuçlanıp sonuçlanmadığını doğrulamak; kaçının içeri alındığı kaçının izolasyona düştüğüne dair analitik panel sağlamaktır.

9.2. Ölçülen Temel Metrikler

```
SELECT 'Total RAW Customers' AS metric, COUNT(*) AS count FROM
stg_customers
UNION ALL
SELECT 'Successfully Loaded', COUNT(*) FROM crm_contacts_clean
UNION ALL
SELECT 'Rejected (Quarantine)', COUNT(*) FROM
crm_contacts_rejected;
```

9.3. Clean / Rejected / Duplicate Sonuçlarının Analizi

METRİK	Toplam	Durum Özeti
RAW Customers	104	Sisteme ithal edilen 1. Dış Kaynak
RAW Leads	101	Sisteme ithal edilen 2. Dış Kaynak
Successfully Loaded	185	Hedef tabloda standart hale getirilen YENİ asıl data
Deduplicated	8	Lead formundan kaynaklandığı tespit edilip engellenme logları
Rejected/Quarantine	12	Sahte olduğu tespit edilerek izole edilen defolu kayıtlar

9.4. ETL Pipeline Başarısının Değerlendirilmesi

Başlangıçtaki Raw toplamı 205 değerini veren (104+101); çıkıştaki operasyonları kapsayan 205 (185+8+12) değerine fire vermeden, son derece senkronize bir tutarlılık sergilemiştir.

10. KARŞILAŞILAN SORUNLAR

10.1. PostgreSQL CTE Kısıtlaması

ETL'nin ilk tasarımı, dönüştürme ve yüklemeyi birleşik blok halinde tek bir Common Table Expression (WITH) üzerinde dizayn etmektir. Ancak PostgreSQL aynı CTE üzerinden INSERT INTO yaparken çoklu ayrı hedefleri kısıtlıyordu.

10.2. TEMP TABLE Yaklaşımına Geçiş

Soruna karşılık, dönüştürülmüş ham verilerin bellek üzerinde süzülmesi için "Temp Table" dizisine yatırılması taktiğine dönülmüştür.

10.3. Çoklu Hedefe Yükleme Probleminin Çözümü

Temp alana istifade edilen veri, arka arkaya satır filtrelerine (WHERE row_num = 1) tabi tutularak parça parça ana hedef tablolara bölünerek INSERT işleminden geçirildi.

10.4. Nihai Mimari Kararın Gerekçesi

Sadece Select yaparak çözülemeyecek çok hedefli (Multi-Destination) bir Insert loglaması, esnek bir veri tabanı programcılığının ve PostgreSQL ara depolama (temp) yapısının gücüyle harmanlanmış gerçek bir mimariye bürünmüştür.

11. SONUÇ

Laboratuvar ortamında ROW_NUMBER() gibi Window fonksiyonlarından, Regex desenlerine varana değin veri kontrol mimarilerindeki filtreleme hızı test ve teyit edilmiştir.

"Bozuk kayıt ne olursa olsun silinir" felsefesinin geçersiz olduğu anlaşılmış, elenmesi gereken ve denetlenmesi gereken tüm "bozuk format" izleri başarılı bir şekilde "Rejected" kalıntı havuzuna aktararak kalitenin şeffaflaştırılması başarılmıştır.

Mükerrer eklentilere dahi Duplicate log listesi gibi profesyonel yapılar vasıtasıyla toleranslı davranılıp silinmemesinin sağlanması kurumsal İş Zekasının (BI) standartlarını kanıtlamıştır.

Veri Temizleme ve ETL süreçlerinin, rastgele ve kontrolsüzce yazılan "script yığınlarıyla" değil; transaction garantisi veren, tekrarlanabilirliği (Idempotency) güvenli ve filtre yetenekleri akademik düzeyde belgelendirilmiş mimarilerden oluşması gerektiği ispat edilmiş ve proje başarıyla noktalanmıştır.

KAYNAKLAR

Ben-Gan, I., Machanic, A., Sarka, D., & Farlee, K. (2015). *T-SQL Querying*. Microsoft Press. (<https://www.microsoftpressstore.com/store/t-sql-querying-9780735685048>)

Fritchey, G. (2018). *SQL Server Execution Plans* (3rd ed.). Redgate Books. (<https://www.red-gate.com/hub/books/>)

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley. (<https://www.wiley.com/en-ie/The%2BData%2BWarehouse%C2%A0ETL%2BToolkit%3A%2BPractical%2BTechniques%2Bfor%2BExtracting%2C%2BCleaning%2C%2BConforming%2C%2Band%2BDelivering%2BData-p-9780764567575>)

Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media. (<https://martin.kleppmann.com/>)

Microsoft Corporation. (2025a). *ROW_NUMBER (Transact-SQL) - SQL Server*. Microsoft Learn. (<https://learn.microsoft.com/en-us/sql/t-sql/functions/row-number-transact-sql?view=sql-server-ver17>)

Microsoft Corporation. (2025b). *SET STATISTICS TIME (Transact-SQL) - SQL Server*. Microsoft Learn. (<https://learn.microsoft.com/en-us/sql/t-sql/statements/set-statistics-time-transact-sql?view=sql-server-ver17>)

Microsoft Corporation. (2025c). *sys.dm_exec_query_stats (Transact-SQL) - SQL Server*. Microsoft Learn. (<https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-exec-query-stats-transact-sql?view=sql-server-ver17>)

Microsoft Corporation. (2025d). *Index Architecture and Design Guide - SQL Server*. Microsoft Learn. (<https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-index-design-guide?view=sql-server-ver17>)

Microsoft Corporation. (2026). *Execution Plan Overview - SQL Server*. Microsoft Learn. (<https://learn.microsoft.com/en-us/sql/relational-databases/performance/execution-plans?view=sql-server-ver17>)

The PostgreSQL Global Development Group. (2026a). *WITH Queries (Common Table Expressions)*. PostgreSQL Documentation. (<https://www.postgresql.org/docs/current/queries-with.html>)

The PostgreSQL Global Development Group. (2026b). *CREATE TABLE*. PostgreSQL Documentation. (<https://www.postgresql.org/docs/current/sql-createtable.html>)